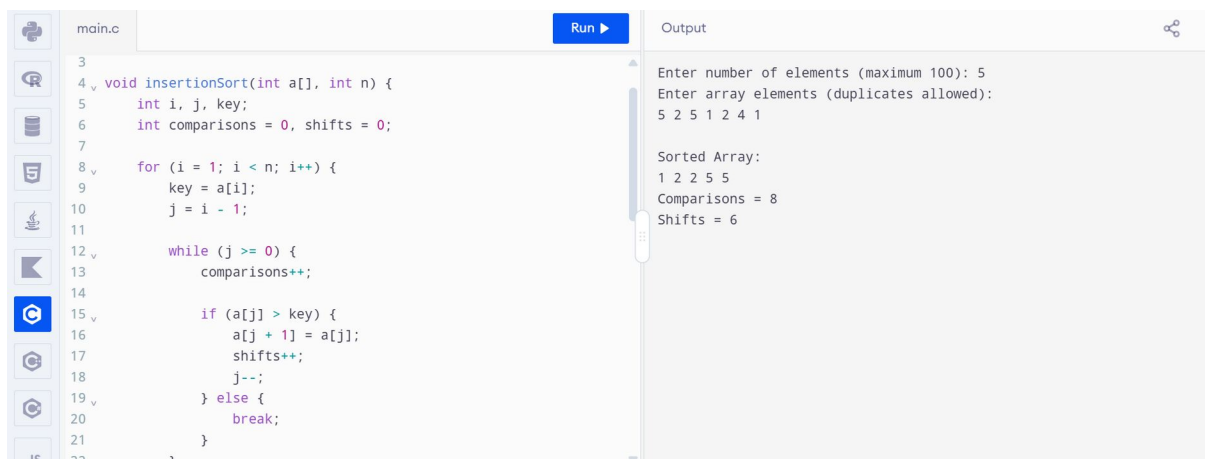


Aim

To implement Insertion Sort on an array containing duplicate elements and arrange them in ascending order while preserving the relative order of equal elements.

Algorithm

1. Read the number of elements and array values, including duplicates.
2. Start from the second element and store it as the key.
3. Compare the key with the elements in the sorted portion on its left.
4. Shift elements greater than the key one position to the right.
5. Insert the key into its correct position.
6. Repeat until all elements are sorted.
7. Display the sorted array and the number of comparisons and shifts



The screenshot shows a C program in a code editor with a 'main.c' tab and a 'Run' button. The code implements Insertion Sort. The output window shows the following text:

```
Enter number of elements (maximum 100): 5
Enter array elements (duplicates allowed):
5 2 5 1 2 4 1

Sorted Array:
1 2 2 5 5
Comparisons = 8
Shifts = 6
```

Complexity Analysis

- Best-case time complexity: $O(n)$, when the array is already sorted.
- Average-case time complexity: $O(n^2)$.
- Worst-case time complexity: $O(n^2)$, when the array is reverse-sorted.
- Auxiliary space complexity: $O(1)$.

Result: Insertion Sort successfully sorts an array containing duplicate elements in ascending order without removing duplicates.